

Documentation Technique

Déploiement de l'Agent Wazuh via Ansible

Version : 1.0

Date : Juin 2026

Plateforme cible : Windows (WinRM)

1. Présentation du projet

Ce projet Ansible automatise le déploiement de l'agent Wazuh sur des machines Windows via le protocole WinRM. Il s'appuie sur un rôle structuré et des variables centralisées pour faciliter la maintenance et la scalabilité.

Wazuh est une plateforme open-source de détection des menaces, d'intégrité des fichiers, de surveillance de la conformité et de réponse aux incidents. L'agent installé sur les endpoints remonte les événements vers le Wazuh Manager pour analyse centralisée.

2. Prérequis

2.1 Environnement de contrôle (nœud Ansible)

- Python 3.x avec pip
- Ansible installé dans un environnement virtuel (ansible-venv)
- Collection pywinrm installée : pip install pywinrm
- Accès réseau aux machines cibles sur le port WinRM (5985 ou 5986)
- ansible-vault pour le chiffrement des mots de passe

2.2 Machines cibles (Windows)

- Windows 10 / Windows Server 2016 et supérieur
- WinRM activé et configuré (HTTP sur le port 5985)
- Compte Administrateur local disponible
- Accès Internet ou dépôt interne pour télécharger le paquet Wazuh

2.3 Versions utilisées

Composant	Version
Wazuh Agent	4.14.5-1
Python	3.x
Ansible	dernière stable dans ansible-venv
WinRM Transport	NTLM

3. Structure du projet

Le projet est organisé selon les conventions standard Ansible :

```
wazuh-ansible/
├── group_vars/
│   └── all.yml          # Variables globales partagées
├── inventory/
│   └── hosts.yml        # Inventaire des machines cibles
├── roles/
│   └── wazuh-agent/
│       ├── defaults/
│       │   └── main.yml  # Valeurs par défaut du rôle
│       ├── tasks/
│       │   ├── main.yml  # Point d'entrée des tâches
│       │   ├── Windows.yml # Tâches spécifiques Windows
│       │   ├── Debian.yml  # Tâches spécifiques Debian
│       │   ├── RedHat.yml  # Tâches spécifiques RedHat
│       │   ├── Linux.yml   # Tâches Linux génériques
│       │   └── macOS.yml   # Tâches spécifiques macOS
│       └── vars/
│           └── main.yml    # Variables calculées du rôle
└── site.yml              # Playbook principal
```

4. Description des fichiers de configuration

4.1 group_vars/all.yml

Ce fichier contient les variables globales applicables à tous les hôtes du projet. Elles définissent les paramètres de connexion au Wazuh Manager.

```
wazuh_manager_address: "10.0.40.3"
wazuh_manager_port: "1514"
wazuh_manager_protocol: "tcp"
wazuh_agent_version: "4.14.5-1"
```

Variable	Valeur	Description
wazuh_manager_address	10.0.40.3	Adresse IP du Wazuh Manager
wazuh_manager_port	1514	Port d'écoute du Manager (par défaut TCP 1514)
wazuh_manager_protocol	tcp	Protocole de communication (tcp ou udp)
wazuh_agent_version	4.14.5-1	Version du paquet agent à installer

4.2 inventory/hosts.yml

L'inventaire déclare les hôtes cibles et leurs paramètres de connexion. Le mot de passe est chiffré avec ansible-vault pour sécuriser l'accès.

```
all:
  children:
    windows:
      hosts:
        win10-test:
          ansible_host: 10.0.10.111
          ansible_user: Administrateur
          ansible_password: !vault | <valeur chiffrée>
          ansible_connection: winrm
          ansible_winrm_transport: ntlm
          ansible_winrm_server_cert_validation: ignore
          ansible_port: 5985
```

Paramètre	Valeur	Description
ansible_host	10.0.10.111	Adresse IP de la machine Windows cible
ansible_user	Administrateur	Compte d'administration Windows local
ansible_password	!vault ...	Mot de passe chiffré via ansible-vault (AES256)
ansible_connection	winrm	Méthode de connexion Ansible pour Windows
ansible_winrm_transport	ntlm	Authentification NTLM pour WinRM
ansible_winrm_server_cert_validation	ignore	Ignore la validation du certificat TLS
ansible_port	5985	Port WinRM HTTP (non chiffré)

Note de sécurité : L'option `ansible_winrm_server_cert_validation: ignore` est acceptable en environnement de lab. En production, utilisez un certificat valide et le port 5986 (HTTPS).

4.3 roles/wazuh-agent/defaults/main.yml

Définit les valeurs par défaut du rôle, surchargeable par l'inventaire ou `group_vars`.

```
wazuh_agent_package_download_path: "/tmp/wazuh-agent"
wazuh_agent_win_package_download_path: "C:\\Temp\\wazuh-agent"
wazuh_agent_package_name: "wazuh-agent-{{ wazuh_full_version }}-{{
wazuh_package_revision }}"
```

Variable	Description
wazuh_agent_package_download_path	Répertoire de téléchargement du paquet sur Linux/macOS
wazuh_agent_win_package_download_path	Répertoire de téléchargement sur Windows (C:\Temp\wazuh-agent)
wazuh_agent_package_name	Nom du paquet construit dynamiquement à partir de la version

4.4 roles/wazuh-agent/vars/main.yml

Variables calculées dynamiquement lors de l'exécution du rôle. Elles dépendent d'un fichier VERSION.json à la racine du playbook.

```
wazuh_version_data: "{{ lookup('file', playbook_dir + '/VERSION.json')
| from_json }}"
wazuh_full_version: "{{ wazuh_version_data.version }}"
wazuh_major_minor_version: "{{
wazuh_version_data.version.split('.')[0:2] | join('.') }}"
wazuh_major_version: "{{ wazuh_version_data.version.split('.')[0] }}.x"
wazuh_package_revision: "1"
wazuh_stage: "{{ wazuh_version_data.stage }}"
local_configs_path: "{{ playbook_dir }}/deployment-config-files"
urls_file: "artifact_urls.yaml"
```

Le fichier VERSION.json doit être présent à la racine du projet et contenir au minimum les champs version et stage.

4.5 site.yml — Playbook principal

Point d'entrée du déploiement. Il applique le rôle wazuh-agent sur le groupe d'hôtes windows en passant les paramètres de connexion au Manager.

```
- name: Déploiement Wazuh Agent - Windows
  hosts: windows
  roles:
    - role: wazuh-agent
  vars:
    wazuh_managers:
      - address: "{{ wazuh_manager_address }}"
        port: "{{ wazuh_manager_port }}"
        protocol: "{{ wazuh_manager_protocol }}"
    wazuh_agent_version: "{{ wazuh_agent_version }}"
```

5. Procédure de déploiement

5.1 Activation de l'environnement virtuel

Avant toute opération, activer l'environnement Python dédié :

```
source ~/ansible-venv/bin/activate
```

L'invite de commande affiche alors (ansible-venv) pour confirmer l'activation.

5.2 Vérification de la connectivité WinRM

Tester la connexion Ansible vers la machine Windows cible avant le déploiement :

```
ansible -i inventory/hosts.yml windows -m win_ping --ask-vault-pass
```

Une réponse pong confirme que WinRM est fonctionnel et que les credentials sont valides.

5.3 Lancement du playbook

```
ansible-playbook -i inventory/hosts.yml site.yml --ask-vault-pass
```

Options utiles :

- `--check` : mode dry-run, simule sans appliquer
- `--limit win10-test` : restreint l'exécution à un hôte
- `-v / -vv / -vvv` : niveaux de verbosité croissants
- `--tags <tag>` : exécute uniquement les tâches taguées

5.4 Gestion du vault

Le mot de passe Administrateur est chiffré avec ansible-vault AES256. Pour modifier la valeur :

```
ansible-vault encrypt_string 'NouveauMotDePasse' --name  
'ansible_password'
```

Remplacer ensuite la valeur dans hosts.yml.

Pour rechiffrer le fichier complet :

```
ansible-vault rekey inventory/hosts.yml
```

6. Architecture réseau

Composant	IP / Réseau	Port	Rôle
Nœud Ansible (PFR-Ansible)	—	—	Orchestrateur Ansible
Machine Windows cible	10.0.10.111	5985 (WinRM HTTP)	Endpoint à déployer
Wazuh Manager	10.0.40.3	1514 (TCP)	Collecteur centralisé des alertes

Le nœud Ansible communique avec les machines Windows sur le port 5985 (WinRM). Une fois l'agent installé, chaque endpoint se connecte au Wazuh Manager sur le port 1514 en TCP.

7. Considérations de sécurité

- Les mots de passe sont systématiquement chiffrés avec ansible-vault (AES256). Ne jamais committer de credentials en clair dans le dépôt.
- WinRM est configuré en HTTP (port 5985) pour cet environnement de lab. En production, basculer vers HTTPS (port 5986) avec un certificat valide et supprimer l'option ignore sur la validation.
- Le transport NTLM offre une authentification sécurisée. Kerberos peut être envisagé dans un domaine Active Directory.
- Restreindre l'accès WinRM aux seules IP du nœud Ansible via les règles de pare-feu Windows.
- Stocker la clé vault dans un gestionnaire de secrets (HashiCorp Vault, AWX, Ansible Tower) pour les déploiements à grande échelle.

8. Dépannage

Symptôme	Cause probable	Action corrective
Unreachable — WinRM connection refused	WinRM non activé ou pare-feu bloquant le port 5985	Exécuter winrm quickconfig sur la machine Windows
Authentication failure (NTLM)	Mauvais identifiants ou vault corrompu	Vérifier le vault avec ansible-vault view
VERSION.json not found	Fichier absent à la racine du playbook	Créer le fichier VERSION.json avec les champs version et stage
Module win_ping not found	pywinrm non installé dans le venv	pip install pywinrm dans ansible-venv
Timeout sur la connexion	Machine cible inaccessible ou IP incorrecte	Vérifier ansible_host dans hosts.yml et la connectivité réseau

9. Évolutions possibles

- Ajouter des groupes d'hôtes supplémentaires (Debian, RedHat, macOS) — les tâches correspondantes sont déjà présentes dans le rôle.
- Intégrer un fichier VERSION.json et artifact_urls.yaml pour gérer les URLs de téléchargement par version.
- Mettre en place un pipeline CI/CD (GitLab CI, GitHub Actions) pour déclencher automatiquement le déploiement.
- Utiliser Ansible Tower ou AWX pour centraliser l'exécution et l'audit des playbooks.
- Ajouter un rôle de vérification post-déploiement vérifiant que l'agent Wazuh est bien démarré et enregistré sur le Manager.

— *Fin du document* —